

Training Neural Networks to Play Connect 4

For this project, our team trained neural networks to play Connect 4 by framing move selection as a 7-class classification problem. Given a board state, the models predict which of the seven columns represents the optimal move.

Data Representation and Preprocessing

Each board was represented as a tensor of shape (6, 7, 2), corresponding to the 6 rows and 7 columns of the Connect 4 board, with separate channels for each player's pieces. This two-plane, one-hot style encoding simplifies the learning task by explicitly separating ownership of pieces, avoiding numeric encodings such as -1, 0, and 1 that would require the network to infer piece identity.

Our labels ranged from 0 to 6, indicating the best column to play. For the CNN, we used a 90/10 training-validation split with a batch size of 256. For the Transformer, we used an 85/15 split with a batch size of 128. With approximately 874,000 total board states, the dataset provided a broad distribution of early-, mid-, and late-game positions, allowing the models to learn diverse spatial and tactical patterns.

CNN Architecture

Our primary model was a deep convolutional neural network implemented in PyTorch. The architecture consisted of four convolutional layers, with channel depth increasing from the 2 input channels to 512 output channels. Each convolution used 3×3 kernels with SAME padding. We found that larger kernels significantly increased parameter count without meaningful validation improvements, while 3×3 filters sufficiently captured local horizontal, vertical, and diagonal alignments.

In early experiments with fewer convolutional layers, validation accuracy plateaued quickly. Adding depth improved performance, suggesting that deeper layers were able to combine local alignment features into higher-level strategic patterns such as compound threats or fork opportunities.

To reduce overfitting, we applied progressively increasing dropout rates from 0.1 up to 0.4 across the convolutional stack. Initial models with minimal dropout achieved high training accuracy but failed to generalize. Increasing dropout stabilized validation performance and narrowed the gap between training and validation accuracy.

After convolutional processing, global average pooling was applied to condense spatial features while preventing memorization of specific board layouts. The pooled representation was passed

through a fully connected layer with 1024 units before the final 7-class output layer. Reducing the size of this decision layer resulted in lower validation accuracy, suggesting that this stage plays an important role in integrating spatial features into a global move preference.

The CNN was trained using cross-entropy loss and optimized with the Adam optimizer at a learning rate of $3e-4$. After 25 epochs, the model achieved a validation accuracy of approximately 63.6%.

Vision Transformer Architecture

In addition to the CNN, we implemented a Vision Transformer (ViT) using TensorFlow/Keras. Rather than leveraging convolutional spatial bias, the Transformer treats each board cell as an individual token.

Each 6×7 board was flattened into 42 tokens (one per cell), with 2 features per token corresponding to the two channels. These tokens were embedded into a 64-dimensional space, and learned positional embeddings were added to preserve spatial structure.

The Transformer architecture consisted of:

- Hidden dimension: 64
- Number of layers: 3
- Number of attention heads: 4
- Dropout rate: 0.2

We selected four attention heads to allow the model to simultaneously capture different relational patterns, such as horizontal, vertical, and diagonal interactions. Lower embedding sizes resulted in worse validation accuracy, suggesting that the model needed enough feature capacity to properly capture relationships between different board cells.

The Transformer took more epochs to train and reached a validation accuracy of 52.05% after 35 epochs. It did not perform as well as the CNN, likely because it does not naturally account for spatial structure. Convolutional layers are designed to capture nearby patterns on a grid, while the Transformer has to learn the board's spatial relationships from positional embeddings instead of having that structure built in.

Training Process

The initial dataset we created contained approximately 350,000 labeled boards. During early experiments, we observed that validation accuracy plateaued relatively quickly, even as training accuracy continued to increase. This divergence suggested that the models were limited by data diversity rather than architecture alone.

To address this, we increased the dataset to about 874,000 board states, which included more mid-game and complex positions. The larger dataset helped reduce overfitting and made validation accuracy more consistent during training. The deeper CNN especially improved with the extra data, suggesting that larger models need more varied examples in order to learn patterns that generalize well.

Both models were trained using cross-entropy loss for 7-class classification. We monitored validation accuracy after each epoch and selected final checkpoints based on peak validation performance to ensure strong generalization.

Model Behavior Analysis

Easily Classified Boards

Best move:	0	Best move:	5
Model move:	0	Model move:	5
Confidence:	1.0	Confidence:	1.0
Correct?:	True	Correct?:	True
<pre> . 0 0 0 X 0 0 . X X 0 0 X X . X 0 X 0 0 0 . 0 X X 0 X X . 0 0 0 X 0 X 0 0 X X 0 X X </pre>		<pre> . 0 . 0 . 0 . . 0 . X X X . . X 0 X X X 0 0 </pre>	

These boards are simple tactical scenarios, often with immediate winning or blocking moves. The model selects the correct column with near 100% confidence in these cases, demonstrating strong recognition of local threats and opportunities.

Ambiguous Boards

Best move:	6	Best move:	6
Model move:	6	Model move:	6
Confidence:	0.332	Confidence:	0.387
Correct?:	True	Correct?:	True
<pre> . . . 0 X 0 0 0 . X . . X 0 X . 0 . . 0 X X . 0 . X 0 X 0 X X </pre>		<pre> X 0 . 0 . . . 0 X . X X . X 0 0 . X 0 . 0 X 0 X 0 X 0 X X 0 0 0 X X 0 </pre>	

More complex boards often contain multiple reasonable moves. In these cases, the model typically predicts the labeled optimal move but with moderate confidence, reflecting uncertainty in positions where strategic tradeoffs exist.

Confident but Incorrect Moves

Best move:	0	Best move:	1
Model move:	1	Model move:	0
Confidence:	0.97	Confidence:	0.999
Correct?:	False	Correct?:	False

```

. . . . . . .
. . . . . 0
. . . . . X
. . . . . X X
. 0 . 0 . 0 0
X 0 . X . 0 X

```

```

. . . . . . .
. . . 0 . . .
. . . X X . .
. . . 0 0 . .
0 . . X 0 . .
X . . 0 X . .

```

Some positions show an important limitation: the model is very confident in the wrong move. These boards usually require thinking several moves ahead or recognizing more complex strategic traps. Since both models only recognize patterns and do not simulate future moves, they have difficulty handling situations that require longer-term planning.

Reflection

This project demonstrated that neural networks can learn strong tactical play in Connect 4 purely through supervised learning on labeled positions. The CNN, benefiting from its spatial inductive bias, proved more effective than the Transformer, which must infer spatial relationships from positional embeddings alone.

Nonetheless, both models are fundamentally pattern recognition systems rather than planning agents. Positions requiring multi-move reasoning or strategic foresight remain challenging, as neither architecture performs search or simulates future states. This limitation explains why validation accuracy remains moderate despite strong performance in tactical scenarios.